

第 2 章 线性表

郑冠杰

2026 年 3 月 2 日

线性表这章的任务和目标

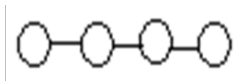
- 熟练 C++ 面向对象的编程方法、C++ 语法、书写完整程序的技巧。
- 接触并逐步熟悉研究一种数据结构的角度和方法。
- 掌握顺序结构的基本功。
- 掌握链式操作的基本功。
- 掌握数据结构工具的建造和工具使用的不同。
- 掌握最基本且简单的线性结构-线性表结构

- 以下哪个操作在单链表上实现比顺序表实现效率高？
 - A. 在最后一个元素的后面插入一个新元素
 - B. 顺序输出前 k 个元素
 - C. 删除所有值为 x 的元素
 - D. 交换表中两个元素

- ① 线性表
- ② 顺序表及实现
- ③ 链表及实现
- ④ 一元多项式
- ⑤ 稀疏矩阵
- ⑥ 字符串 *

线性结构定义

- ▶ 一组特征相同且数量有限的元素构成的集合。该集合可以为空，也可以不为空。
- ▶ 当不为空时，有唯一一个元素被称为首元素，有唯一一个元素被称为尾元素。
- ▶ 除了尾元素，每个元素有且仅有一个直接后继元素；除了首元素，每个元素有且仅有一个直接前驱元素。



常见的几种线性结构

线性表 (List)、时间有序表 (Chronological Ordered List)、排序表 (Sorted List)、频率有序表 (Frequency Ordered List) 等。

线性表：通过元素之间的相对位置来确定它们之间相互关系。

时间有序表：按元素到达结构的时间先后，确定元素之间的关系。栈/队列

排序表：据元素的关键字值来确定其间的关系。

频率有序表：按元素的使用频率确定它们之间的相互关系。

一种仅由元素的相互位置确定它们之间相互关系的线性结构，元素之间呈现出你先我后的关系。

线性表的**规模或长度**是指线性表中元素的个数。

特别地：当元素的个数为零时，该线性表称为空表。

线性表 List 的 ADT：描述关系和关系操作

Data: $\{x_i | x_i \in ElemSet, i = 1, 2, 3, \dots, n, n > 0\}$ 或 Φ ; ElemSet 为元素集合。

Relation: $\{< x_i, x_{i+1} > | x_i, x_{i+1} \in ElemSet, i = 1, 2, 3, \dots, n - 1\}$, x_1 为首元素, x_n 为尾元素。

Operations:

initialize 输入：无或指定 List 的规模。 输出：分配相应空间及初始化。

isEmpty 输入：无 输出：表 List 为空返回 true, 否则返回 false。

isFull 输入：无 输出：表 List 为满返回 true, 否则返回 false。

length 输入：无 输出：返回表 List 中的元素个数。

get 输入：已知元素序号。 输出：如果该序号元素存在, 则返回相应元素的数据值。

find 输入：已知元素的数据值。 输出：查找成功, 返回元素的序号, 否则返回查找失败标志。

线性表 List 的 ADT：描述关系和关系操作

insert 输入：已知待插入的元素及插入位置。 输出：如果插入位置合理，在指定位置插入该元素。

remove 输入：已知被删元素的序号。 输出：删除该元素。

clear 输入：无 输出：删除表 List 中的所有元素。

常见的基本操作来源于生活中对这种结构的了解。

基本操作可以分为几大类型：

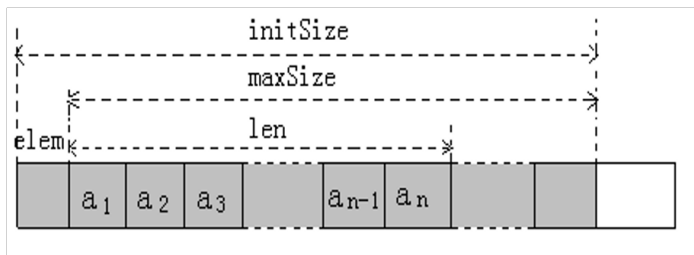
结构类、属性类、数据操纵类、遍历类和典型应用类

线性表

- ① 线性表
- ② 顺序表及实现
- ③ 链表及实现
- ④ 一元多项式
- ⑤ 稀疏矩阵
- ⑥ 字符串 *

- ▶ 元素存放在内存中一块连续的空间里。
- ▶ 借助存储空间物理上的连续性，元素可以按照其逻辑顺序依次存放，即元素存放的物理顺序和它的逻辑顺序是一致的。
- ▶ 顺序存储的线性表称顺序表。
- ▶ 在高级语言的固有数据类型中，数组在存储器中就表现为一块连续的空间，因此用数组实现顺序表非常合适。
- ▶ 数组中各元素位置由其下标来表示，它同时也是相应元素的位置序号。

顺序表的存储映像图



存储映像图

$elem$ 为数组名字，数组存储线性表。

$maxSize$ 为 len 的上界； $initSize$ 为最大的存储空间数， $maxSize = initSize - 1$

$elem[0]$ 用于其它特殊用途，如果不用于特殊用途 $maxSize = initSize$

len 为元素个数，即顺序表长度；

顺序表 seqList 及操作的定义 (seqList.h)

seqList

```
#include <iostream>
#define INITSIZE 100
using namespace std;

//用于异常处理中识别错误类别
class illegalSize{};
class outOfBound{};
```

```
template <class elemType>
class seqList
{private:
    elemType *elem; // 顺序表存储数组，存放实际数据元素。
    int len;        // 顺序表中的元素个数，亦称表的长度。
    int maxSize;    // 顺序表的最大的最大可能的长度。
    void doubleSpace(); //私有函数，做内部工具
```

顺序表 seqList 及操作的定义 (seqList.h)

seqList

```
public:
    seqList(int size=INITSIZE);
    //初始化顺序表
    //表为空返回 true, 否则返回 false。
    bool isEmpty()const { return ( len == 0 )
        ; }
    //表为满返回 true, 否则返回 false。
    bool isFull()const { return (len ==
        maxSize); }
    int length()const {return len;}
    //表的长度, 即实际存储元素的个数。
```

注意各处: const 和 const & 的用法

```
    elemType get(int i )const;
    //返回第 i 个元素的值
    //返回值等于 e 的元素的序号, 无则返回 0.
    int find (const elemType &e )const;
    //在第 i 个位置上插入新的元素 (值为 e),
    //使原来的第 i 个元素成为第 i+1 个元素。
    void insert(int i, const elemType &e );
    // 若第 i 个元素存在, 删除并将其值放入 e 指向
    // 的空间。
    void remove(int i);
    void clear() { len=0; }; //清除顺序表, 使
    // 得其成为空表
    ~seqList() { delete []elem; }; //释放表占
    // 用的动态数组
};
```

i 和 e 用法的区别

顺序表基本操作的实现代码 (seqList.h)

seqList

```
//构造函数任务是给属性赋初值，
//注意模板函数用法：帽子和胡须
template <class elemType>
seqList<elemType>::seqList(int size)
//初始化顺序表
{
    elem = new elemType[size]; //申请动态数组
    if (!elem) throw illegalSize();
    //连续空间，判定申请是否成功
    maxSize = size-1;
    //0下标位置用于查找时做哨兵位。
    len = 0;
}
```

```
template <class elemType>
void seqList<elemType>::doubleSpace()
{
    int i;
    elemType *tmp = new elemType[2*maxSize];
    if (!tmp) throw illegalSize();
    //比较return

    for (i=1; i<=len; i++)
        tmp[i] = elem[i];

    delete []elem;    elem = tmp;
    maxSize = 2*maxSize - 1;
}
```

顺序表基本操作的实现分析

查找操作：待查元素放哨兵位，从后往前比较

摆龙门阵：

哨兵单元		查找 58 比较 4 次				
58	36	14	58	20	42	15
			↑	↑	↑	↑
哨兵单元		查找 72 比较 7 次				
72	36	14	58	20	42	15
↑	↑	↑	↑	↑	↑	↑

分析时间效率：**平均**

查找成功情况：

查找每个位置元素等概率 $1/n$

平均： $(1+2+\cdots+n)1/n = (n+1)/2$

时间复杂度： $O(n)$

查找不成功情况：

每次查找从尾部到哨兵位，比较 $n+1$ 次

时间复杂度： $O(n)$

顺序表基本操作的实现代码

find

```
template <class elemType> //注意各处const, const+&组合的用法
int seqList<elemType>::find (const elemType &e )const
// 返回值等于e的元素的序号, 无则返回0.
{
    int i;
    elem[0] = e; //哨兵位先置为待查元素
    for (i=len; elem[i]!=e; i--);
    return i;
}
```

顺序表基本操作的实现分析

插入操作：新元素欲插在下标为 i 的位置， i 可能的取值为 $n+1, n, \dots, 2, 1$ 。

摆龙门阵：

哨兵单元 第3个位置上插入9

	36	14	58	→ 20	→ 42	→ 15	→
--	----	----	----	------	------	------	---

15, 42, 20, 58依次向后移动一个单元

哨兵单元	插入后状态
------	-------

	36	14	9	58	20	42	15
--	----	----	---	----	----	----	----

特别注意：是从后往前至 i 位置逐个元素后移一位。

分析时间效率：

插入每个位置等概率 $1/(n+1)$

移动的次数分别为 $0, 1, 2, \dots, n$

平均: $(1+2+\cdots+n)/(n+1)=n/2$

时间复杂度: $O(n)$

如何写出一个完整的程序？

“五步口诀法”：

- ① 参数检查
- ② 空间是否支持
- ③ 核心操作
- ④ 对其他属性的影响
- ⑤ 正确返回

顺序表基本操作的实现代码

insert

```
template <class elemType>
void seqList<elemType>::insert (int i, const elemType &e )
{   int k;
    if ((i<1)|| (i>len+1)) return; //插入位置越界
    if (len==maxSize) doubleSpace(); //空间满了, 无法插入元素

    for (k=len+1; k>i; k--)
        elem[k]=elem[k-1];
    elem[i]=e;
    len++;
}
```

看帽子：参数类型

警惕：合法而不合理的情况

易错点—循环检查

左右边界的检查

即 k 的第一次和最后一次满足循环条件的取值。

顺序表基本操作的实现分析

删除操作：待删元素在下标为 i 的位置， i 可能的取值为 $n, \dots, 2, 1$ 。

摆龙门阵：

哨兵单元 删除第3个元素

	36	14	58	←	20	←	42	←	15
--	----	----	----	---	----	---	----	---	----

向前移动次序：20, 42, 15

哨兵单元 删除后状态

	36	14	20	42	15	15
--	----	----	----	----	----	----

特别注意：是自 $i+1$ 位置开始，从前往后逐个元素前移一位。

分析时间效率：

删除每个位置等概率 $1/n$

移动的次数分别为 $0, 1, 2, \dots, n-1$

平均： $(1+2+\dots+n-1)/n = (n-1)/2$

时间复杂度： $O(n)$

自行书写程序 5 分钟

顺序表基本操作的实现代码

remove

```
template <class elemType> //注意五步口诀法
void seqList<elemType>::remove (int i)
{   int k;
    if ((i<1)|| (i>len)) return;

    for (k=i; k<len; k++)
        elem[k]=elem[k+1];
    len--;
}
```

get

```
template <class elemType>
elemType seqList<elemType>::get(int i) const //
    返回第i个元素的值
{
    if ((i<1)|| (i>len)) throw outOfBound();
    return elem[i];
}
```

顺序表基本操作的实现代码

常见错误:

1. 混淆 `len` 和 `maxSize` 的含义, 前者是实际元素的个数, 后者是存储空间的大小, 也是最多能存多少元素的限制。
2. `seqList` 对象作为函数形参, 直接用对象形式而不用引用形式, 这样即浪费空间又引起 `seqList` 类的复制构造函数的执行, 降低运行效率。
3. `insert` 函数实现中忘记检查位置 `i` 的合理性、忘了检查表中是否有空间可以支持插入一个新的元素等, 造成算法不完整。可以试着使用分析参数、空间检查、核心操作、对其他属性的影响、正确返回的“五步口诀法”, 就可以设计出一个相对完整的程序。

顺序表应用—基本操作的测试 main.cpp

main

```
#include <iostream>
#include "seqlist.h"
using namespace std;

//求两个正整数集合的交集，用线性表处理集合问题。
int main()
{
    seqList<int> list1(20), list2(20), list3
        (20); //实例化对象
    int i, j, x;
    int len1, len3;
```

```
//输入第一个整数集合中的元素，输入零结束：
i=1;
cout<<"输入第一个正整数集合，以零为结束标志：";
cin>>x;

while (x!=0)
{
    list1.insert(i,x);
    i++;
    cin>>x;
}
```


顺序表应用—基本操作的测试 main.cpp

main

```
//输入第二个整数集合中的元素，输入零结束：
i=1;
cout<<"输入第一个正整数集合，以零为结束标志： ";
cin>>x;

while (x!=0)
{
    list2.insert(i,x);
    i++;
    cin>>x;
}
```

```
//求list1,list2的交集，结果存入list3
len1 = list1.length();
j=1;
for (i=1; i<=len1; i++)
{
    x=list1.get(i);
    if (list2.find(x)!=0)
    {
        list3.insert(j,x);
        j++;
    }
}
```

顺序表应用—基本操作的测试 main.cpp

main

```
//显示 list3 中的元素
cout<<"两个集合的交集元素为: ";
len3 = list3.length();
for (i=1; i<=len3; i++)
{
    x=list3.get(i);
    cout<<x<<" ";
}
cout<<endl;
return 0;
}
```

后续问题：程序的调试—黑匣子？
用中间变量输出法和 debug 工具の場合

总结

1. 线性结构定义、4 种常见的线性结构、线性表
2. 线性表相关的概念：表长、空表、首尾元素、前驱后继
3. 线性表的基本操作：3 大类 — 逻辑结构和基本操作成果是 ADT
4. ADT 描述元素集合、元素关系、基本操作
5. 线性表的顺序存储-线性表（连续的空间、空间先后表达元素先后）— 成果是存储映像图，
进而是类模板

6. 基本操作实现中

- ✓ 都是对类属性做点什么
- ✓ 构造函数给属性赋初值，通常是 0 个元素状态。
- ✓ 具体算法映像图上举例摆龙门阵、获得核心操作
- ✓ 五步口诀法写完整程序，参数、空间、核心、其他属性、返回

7. 基本操作实现效率

- A) 查找、插入、删除为 $O(n)$, 其余 $O(1)$
- B) 插入和删除引起大量数据移动

演示：如何规划 project 中的模块

- ✓ 常规应该是 seqlist.h seqlist.cpp main.cpp 三个文件在同一个 project 中
 - ✓ 因为有类模板，以上存在无法链接到类中成员函数的问题
变通方法：
 - ① seqlist.cpp 取消，内容加到 seqlist.h 中
 - ② seqlist.cpp 不取消，但 main.cpp 中除了要 include seqlist.h, 还要 include seqlist.cpp
- oj 上则 seqlist.h 和 seqlist.cpp 全部取消，都放在 main.cpp 中。

线性表

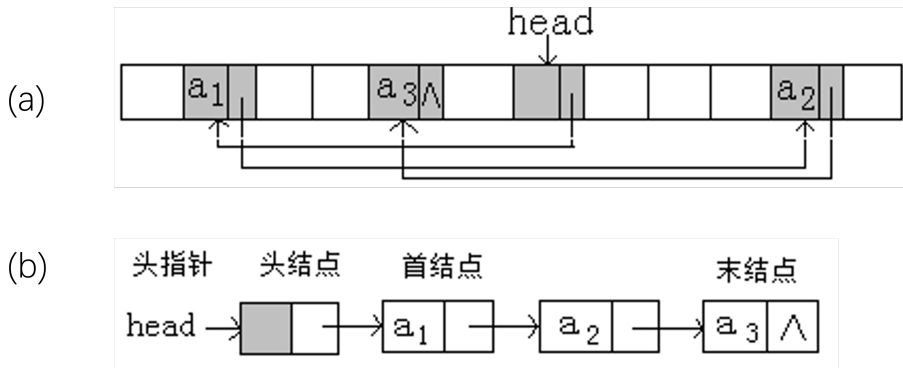
- ① 线性表
- ② 顺序表及实现
- ③ 链表及实现
- ④ 一元多项式
- ⑤ 稀疏矩阵
- ⑥ 字符串 *

顺序表插入、删除时间代价的分析，可以看出其时间复杂度是**线性阶**的，而且会引起已存储元素的**大规模位置移动**。

改进方法：链式结构

- ▶ 各个元素的物理存放在存储器中是任意的，**不一定连续**。
- ▶ 每个元素放在一个独立的存储单元中，元素间的逻辑关系依靠存储单元中**附加指针**来完成。
- ▶ 采用链式存储结构存储的线性表，称为**链表**。

链表的存储映像图



面临：结构体和指针的挑战！

为了清晰看出逻辑关系，以后链表用图 (b) 来表示。

结构类型，结构变量，结构指针

也称：类，对象，指针

linkList

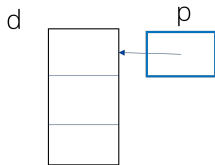
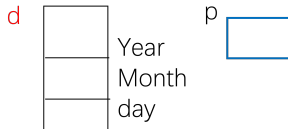
```
struct dateT
{
    int year;
    int month;
    int day;
};

class dateT
{
public:
    int year;
    int month;
    int day;
};

{
    dateT d, *p;

    d.year = 2020;
    d.month = 3;
    d.day = 11;

    p = &d;
    cout<<p->year<<endl;
    cout<<p->month<<endl;
    cout<<p->day<<endl;
}
```



考虑：d、p 空间如何释放

结构类型，结构变量，结构指针

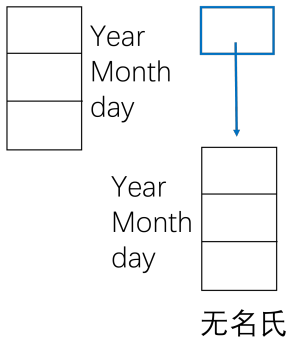
linkList

```
struct dateT
{
    int year;
    int month;
    int day;
};

class dateT
{
public:
    int year;
    int month;
    int day;
};

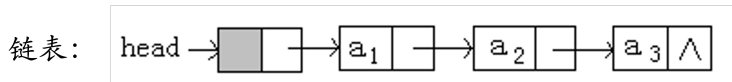
{
    dateT d, *p;
    p = new dateT;

    p->year = 1010;
    p->month = 1;
    p->day = 1;
    delete p;
    p->year = 2021; //非法
    p = &d; //合法
}
```



考虑：d、p、无名氏空间如何释放

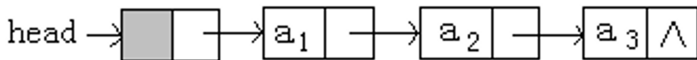
结点和链表



结点：定义一个类模板

单链表特点

- ▶ 头指针 head 指向了头结点。
- ▶ 头结点并不是线性表中的一部分，它的指针字段 next 给出了首结点的地址。
- ▶ 线性表中最后一个结点的指针字段 next 的值为 NULL。
- ▶ 顺着头指针 head，可以很方便地逐个访问单链表中的所有结点。



头指针 头结点 首结点

通常Head在无头结点的情况下仍叫头指针

单链表类

- ▶ 它的任何一个结点包含了一个存储元素数据值的字段和一个存储该结点的直接后继结点地址的指针字段。
- ▶ 提供一个单链表只需要给出头结点的地址即头指针。
- ▶ 单链表结点类定义、单链表类定义。



链表：仅一个描述因子

单链表结点类：(linkList.h)

linkList

```
class outOfBound{};
template <class elemType>
class linkList; //类的前向说明
template <class elemType>
class node
{
    friend class linkList<elemType>;
private:
    elemType data;
    node *next;
```

```
public:
    node():next(NULL){};
    node(const elemType &e, node *N=NULL)
    { data = e; next = N; };
};
```

如何决定设计哪些构造函数？

单链表类：(linkList.h)

linkList

```
template <class elemType>
class linkList
{
private:
    node<elemType> *head;
public:
    linkList(); //构造函数，建立一个空表
    bool isEmpty ()const; //表为空返回true，否则返回false。
    bool isFull ()const {return false;}; //表为满返回true，否则返回false。
    int length ()const; //表的长度
```

```
};

elemType get(int i)const; //返回第i个元素的值
//返回值等于e的元素的序号，从第1个开始，无则返回0.
int find (const elemType &e )const;
//在第i个位置上插入新的元素（值为e）。
void insert (int i, const elemType &e );
//若第i个元素存在，删除并将其值放入e指向的空间。
void remove (int i, elemType &e);
void reverse(); //元素就地逆置
void clear (); //清空表，使其为空表
~linkList();
```

链表基本操作的实现代码 (linkList.h)

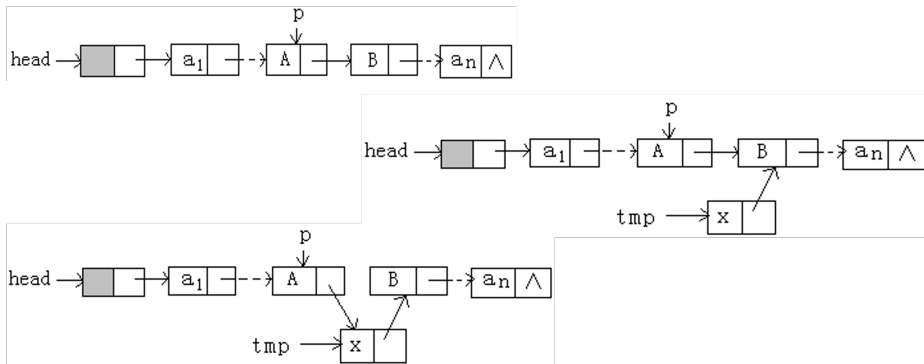
linkList

```
template <class elemType>
linkList<elemType>::linkList() //构造函数，建立一个空表
{
    head = new node<elemType>();
}
template <class elemType>
bool linkList<elemType>::isEmpty ()const //表为空返回 true，否则返回 false。
{
    if (head->next==NULL) return true;
    return false;
}
```



链表基本操作的实现分析

插入操作：将元素 x 插入在 P 指针所指结点之后



摆龙门阵！找核心操作

链表基本操作的实现分析

插入总结：遵循“创建空间、武装自己，融入队伍”

- ① 在内存中创建新结点。
- ② 武装新结点：将 x 写入新结点的 `data` 字段， p 指针所指结点的下一结点地址写入新结点的 `next` 字段，使 p 所指结点的下一结点成为新结点的直接后继结点。
- ③ 将新结点地址写入 p 的 `next` 字段，使新结点成为 p 所指结点的直接后继结点。

链表基本操作的实现分析

插入具体语句：

```
tmp = new node<elemType>();  
tmp->data = x;  
tmp->next = p->next;  
p->next = tmp;
```

或者：

```
tmp = new node<elemType>(x, p->next);  
p->next = tmp;
```

或者：一个四合一语句

```
p->next = new node<elemType>(x, p->next);
```

时间复杂度分析：

当 P 已经指向了插入位置的前一个结点时，插入操作和结点个数无关，时间复杂度为 $O(1)$ 。

但 P 要指向某个顶点时间复杂度 $O(n)$

链表基本操作的实现代码

insert

```
template <class elemType>
void linkList<elemType>::insert (int i, const elemType &e )
//在第 i 个位置上插入新的元素 (值为 e) 。
{
    if (i<1) return; //参数 i 越界
    int j=0; node<elemType> *p=head;

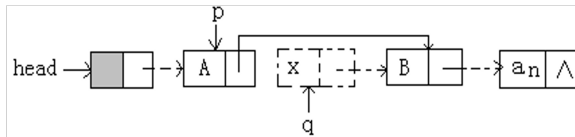
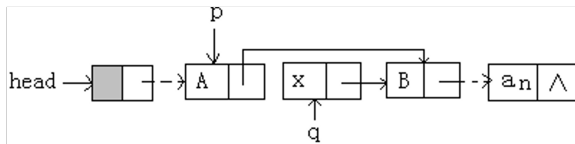
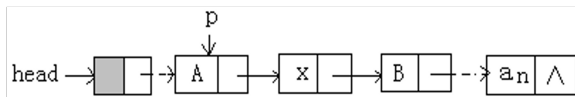
    while (p&& j<i-1) //注意双条件的限制
    { j++; p=p->next; }

    if (!p) return; //参数 i 越界
    p->next = new node<elemType>(e, p->next);
}
```

五步口诀法!

链表基本操作的实现分析

删除操作：删除 P 指针所指结点之后的那个结点



摆龙门阵！找核心操作

链表基本操作的实现分析

删除总结：

- ① 记住待删除结点地址。
- ② 将值为 x 的结点旁路掉。
- ③ 回收原本存储 x 的结点占用的空间。

具体语句：

```
node *q=p->next;  
p->next = q->next;  
delete q;
```

时间复杂度分析：

当 P 已经指向了待删除结点的前一个结点时，删除操作和结点个数无关，时间复杂度为 $O(1)$ 。

$p \rightarrow next$ 在等号左右的区别

链表基本操作的实现代码

remove

```
template <class elemType>
void linkList<elemType>::remove (int i,
    elemType &e)
{  if (i<1) return;
    node<elemType> *p ;  p=head;
    int j=0;

    while (p && j<i-1){  p=p->next; j
        ++;}
    if (!p) return;
```

```
        node<elemType> *q ;
        q = p->next;
        if (!q) return;
        e = q->data;
        p->next = q->next;
        delete q;
    }
```

循环检查：左右边界
链表检查：-> 处

易错点：少了 if (!q) return;, 非法访问了空间。
处理方法：检查->

链表基本操作的实现分析

查找操作：

找值为 x 的结点，顺首结点逐个向后检查、匹配。

单链表和顺序表中，时间复杂度都是 $O(n)$ 。

找第 k 个结点，顺序表 $O(1)$ ，链表 $O(n)$ 。

其他基本操作：

isFull：因每次只申请一个结点空间，故总为 false。

clear：除了头结点删除并释放整个单链表中结点，回到初始化后的状态。

链表基本操作的实现代码

length

```
template <class elemType>
int linkList<elemType>::length ()const //表的
    长度
{
    int count=0;
    node<elemType> *p;
    p=head->next;
    while(p)
    {    count++;    p=p->next;    }
    return count;
}
```

get

```
template <class elemType> //注意： 五步口诀法
elemType linkList<elemType>::get(int i )const
//返回第 i 个元素的值,首元素为第 1 个元素
{
    if (i<1) throw outOfBound();
    int j=1;
    node<elemType> *p = head->next;
    while (p&& j<i) {p=p->next; j++;}
    if (p) return p->data;
    throw outOfBound();
}
```

链表基本操作的实现代码

find

```
template <class elemType>
int linkList<elemType>::find (const elemType &e )const
//返回值等于e的元素的序号，从第1个开始，无则返回0.
{   int i=1;
    node<elemType> *p = head->next;

    while (p)
    {   if (p->data==e) break;
        i++;   p=p->next;
    }
    if (p) return i;
    return 0;   }
```

条件的顺序很重要

While (p && p->data !=e)

{ p=p->next; i++ }

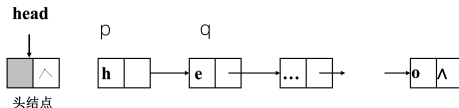
两种常用技巧：兄弟协同法、首席插入法

clear

```
template <class elemType>    //P、Q兄弟协同法
void linkList<elemType>::clear () //清空表，使其为空表
{
    node<elemType> *p,*q;

    p=head->next;      head->next=NULL;

    while (p)
    {
        q=p->next;
        delete p;
        p=q;
    }
}
```



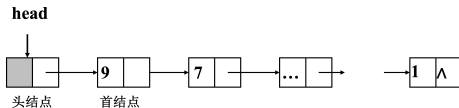
两种常用技巧：兄弟协同法、首席插入法

insert

最速插入位置—脖子//首席插入法

```
template <class elemType>
void linkList<elemType>:: insert(const elemType a[], int n)
{ node *tmp,
  for (int i=0; i<n; i++)
  { tmp = new node(a[i], head->next); head->next = tmp; }
}
```

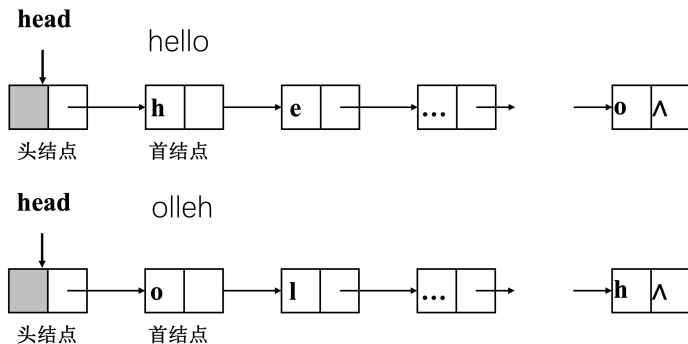
或者 `head->next = new node(x, head->next);`



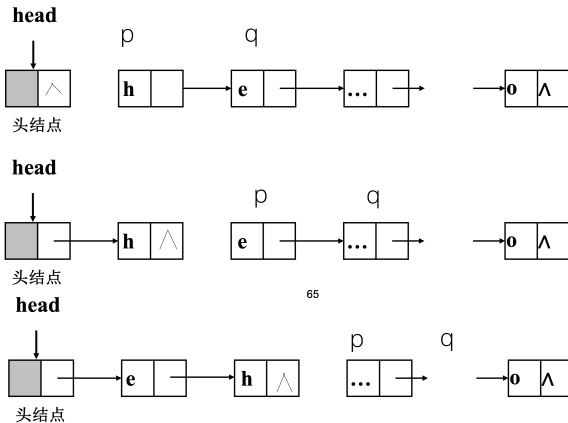
`a[5]=1,3,5,7,9`

练习：对一个单链表进行就地逆置—摆龙门阵

存储映像图：



练习：对一个单链表进行就地逆置—摆龙门阵



65

“兄弟协同法” + “首席插入法” 实现单链表的就地逆置

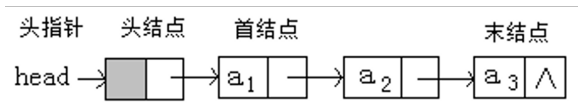
reverse

```
template <class elemType>
void linkList<elemType>::reverse()
{
    node<elemType> *p,*q; //兄弟俩协同
    p=head->next;    head->next = NULL;
    while (p)
    {
        q = p->next;
        p->next = head->next; head->next = p; //首席插入
        p=q;
    }
}
```

易错点：心口不一

常见错误

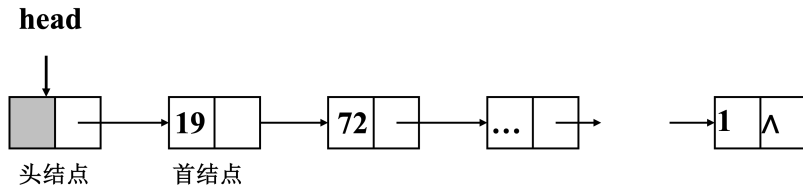
- ▶ 指针 p 未被初始化或者为空，读取其指向的字段如 $p \rightarrow data$ ，如循环检查 p 所指的结点中值是否 x ，可用 $p = head \rightarrow next$; while ($p \ \&\& \ p \rightarrow data \neq x$) $p = p \rightarrow next$ 。
- ▶ 插入时， p 首先指向了首结点 $p = head \rightarrow next$ ，应该指向头结点 $p = head$ 。



- ▶ 原本指向了一个结点，但其指向的结点空间已经释放，仍要读取其所指结点的字段。如 $p = head$; delete p ; $p = p \rightarrow next$;
 $p \rightarrow next$ 非法访问了不能访问的内存空间。

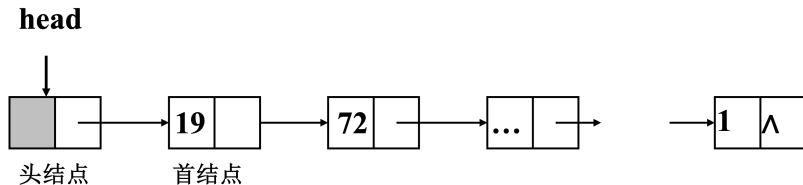
练习

练习：输出链表中倒数第 K 个元素的值



练习

练习：输出链表中倒数第 K 个元素的值



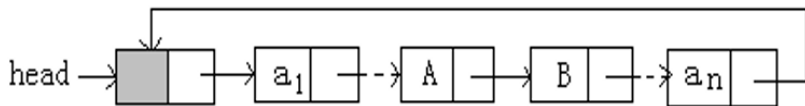
思路讨论？

发现三种：

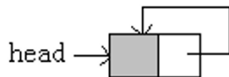
- ① 数组
- ② 数完，算 $n-k$
- ③ 两个指针一前一后

单循环链表

末结点的 next 不再指向空，而是指向头结点。



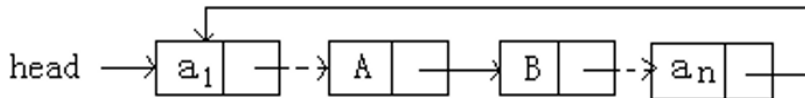
空单向循环链表，只有头结点。



优点：从表中任何一个结点出发，都可以顺 next 指针访问到所有结点。

不带头结点的单循环链表

为了循环方便，不带头结点的单循环链表居多，head 直接指向首结点。



空单向循环链表，head 指向空。

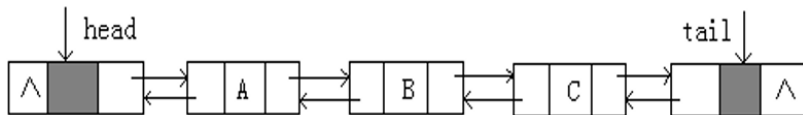
head $\longrightarrow$ NULL

典型应用：约瑟夫环

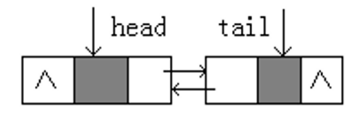
比较普遍的是用 rear 指针，而不用 head 指针

双向链表

每个结点有 `prior` 和 `next` 两个指针，分别指向直接前驱和直接后继结点。



空双向链表，只有头、尾结点。

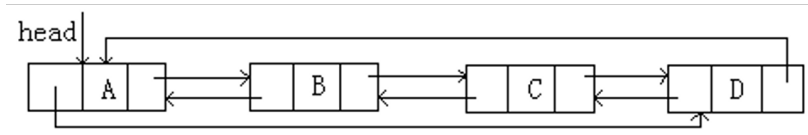


优点：根据待查元素在前或后半段，决定自 `head` 向后还是自 `tail` 向前。

需要一个参数 `length` 来辅助判断，优势才能凸显。

双向循环链表

不带头、尾结点的双向循环链表。

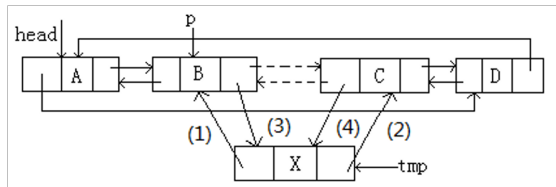


空双向链表。

head $\longrightarrow$ NULL

双向链表

插入： 将元素 x 插入到 p 指针所指结点之后。
如果新结点插入在首结点位置，操作又有不同，请课后练习。



```
node *tmp = new node() // (1)
tmp->data = x; // (2)
tmp->prior = p;
tmp->next = p->next;
tmp->prior->next = tmp; // (3)
if (tmp->next)
    tmp->next->prior = tmp; // (4)
```

顺序、链表结构各自的优势和劣势：

顺序结构：

- ① 找第 i 个元素方便，时间复杂度 $O(1)$
- ② 数组操作特别熟练，不易错
- ③ 需要连续空间，且要预估空间
- ④ 查找、插入和删除时间效率 $O(n)$

链式结构：

- ① 找第 i 个元素不方便，时间复杂度 $O(n)$
- ② 链表操作不熟练，易出错
- ③ 不需要连续空间，也不要预估空间
- ④ 查找、插入和删除时间效率虽然也是 $O(n)$ ，但无数据大规模移动

线性表

- ① 线性表
- ② 顺序表及实现
- ③ 链表及实现
- ④ 一元多项式
- ⑤ 稀疏矩阵
- ⑥ 字符串 *

一元多项式

在数学上, 一元多项式一般表示为如下形式:

$$p_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

在计算机内实现时, 可以用线性表来表示:

$p = (p_0, p_1, p_2, \dots, p_i, \dots, p_n)$, 其中结点 $p_i (0 \leq i \leq n)$ 表示幂为 i 项的系数。

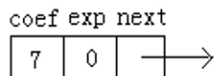
一元多项式存储方法

- ▶ 一种方法是：为了表示系数和项的对应关系。
 i 次幂项的系数 p_i 存放在下标为 i 的数组结点中，即便 p_i 为 0，相应的数组分量也不能挪作它用。
- ▶ 另外一种处理方法是：只存储系数不为 0 的项，每一项除了存储它的系数，还要存储它的幂。
两个多项式的加法处理起来比第一种方法复杂。
用数组时，要预估一个多项式的规模，分配足够的空间。

一元多项式的链式存储

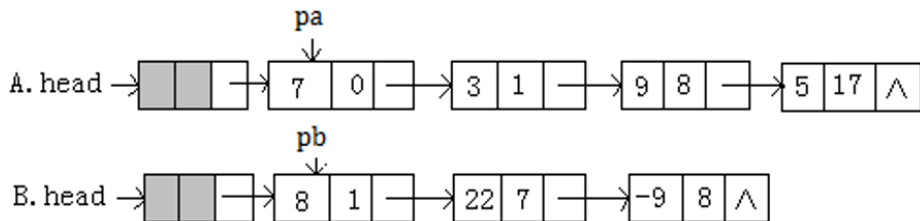
- ▶ 每个结点存放一元多项式中的一项的信息。信息包括该项的系数和幂，零系数项不予存储。
- ▶ 多项式的项数是不确定的，链式存储的好处是多项式的项数可以动态地增长，不存在溢出问题。
- ▶ 用单链表表示一元多项式。在存储实现时，按照幂由小到大的原则进行，这样该单链表便成为幂有序的单链表。
- ▶ 链表中的结点，包含两个部分：数据部分和指针部分。
- ▶ 数据部分又包含了系数 `coef` 和幂 `exp` 二个字段。

一元多项式的链式存储



多项式如: $A = 7 + 3x + 9x^8 + 5x^{17}$

$$B = 8x + 22x^7 - 9x^8$$



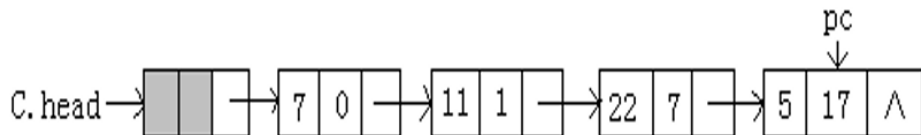
两个一元多项式相加

对 pa 和 pb 所指结点，反复执行如下操作，直至其中一个单链表中的结点全部读取完毕。

- ▶ **幂指数相等**：如果这二个结点的系数之和为零，和式中不增加项，否则按照相加后的系数、相应幂指数创建一个新结点，作为和式单链表 C 的末结点， pa 、 pb 后移。
- ▶ **指针 pa 幂指数小**：按 pa 指向的结点的系数、幂指数创建一个新结点作为单链表 C 的末结点， pa 后移， pb 不变。
- ▶ **指针 pb 幂指数小**：按 pb 指向的结点的系数、幂指数创建一个新结点作为单链表 C 的末结点， pb 后移， pa 不变。

两个一元多项式相加

将非空多项式单链表 (可能是 A 的单链表, 也可能是 B 的单链表) 中的剩余结点, 按序逐个创建新结点插入在单链表 C 的尾部。



多项式 Polynomial 及其部分基本操作的声明、定义 (polynomial.h)

Polynomial

```
#ifndef POLYNOMIAL_H_INCLUDED
#define POLYNOMIAL_H_INCLUDED
#include "linklist.h"
using namespace std;

struct Type
{   int coef;    // 系数
    int exp;     // 幂指数
}

template <class elemType>
struct Node
{   elemType data;
    Node* next;
};
```

```
template <class elemType>
struct Polynomial
{
    private:
        Node<elemType>* head;
        elemType stop_flag; // 用于判断多项式输入
                               结束。
    public:
        //从用户处获取结束标志并初始化多项式
        Polynomial(const elemType &stop);
        void getPoly(); //读入一个多项式。
        void addPoly(const Polynomial &L1, const
                      Polynomial &L2);
        // L3=L1+L2。
        void dispPoly(); //显示一个多项式
        void clear(); //释放多项式空间
        ~Polynomial(){clear(); delete head;};
};
```


多项式 Polynomial 及其部分基本操作的声明、定义 (polynomial.h)

Polynomial

```
// getStop为外部函数，即非类成员函数
template <class elemType>
void getStop(elemType &stopFlag) //从用户
    处获取结束标志
{
    int c,e;
    cout<<"请输入系数、指数对作为结束标
        志,如(0,0): ";
    cin>>c>>e;
    stopFlag.coef = c;
    stopFlag.exp = e;
}
```

```
template <class elemType>
Polynomial<elemType>::Polynomial(const elemType &
    stop) //初始化多项式
{
    head = new Node<elemType>();
    stop_flag.coef = stop.coef;
    stop_flag.exp = stop.exp;
}

template <class elemType>
void Polynomial<elemType>::getPoly() //读入一个多
    项式
{
    Node<elemType> *p, *tmp;
    elemType e;
```

多项式 Polynomial 及其部分基本操作的声明、定义 (polynomial.h)

Polynomial

```
p=head;
cout<< “请按照指数从小到大输入系数、指数对, ”
      << “最后输入结束标志对结束: \n” ;
cin>>e.coef>>e.exp;

while (true)
{
    if ((e.coef==stop_flag.coef)
        &&(e.exp==stop_flag.exp)) break;
```

```
    tmp = new Node<elemType>();
    tmp->data.coef = e.coef;
    tmp->data.exp = e.exp;
    tmp->next = NULL;
    p->next = tmp;
    p=tmp;

    cin>>e.coef>>e.exp;
}

template <class elemType>
void Polynomial<elemType>::addPoly(const Polynomial &
    La,const Polynomial &Lb)// La+Lb.
```

{- - -见教材- - -}

多项式 Polynomial 相加主程序 (main.cpp)

main

```
#include <iostream>
#include "polynomial.h"
using namespace std;
int main()
{
    Type stop_flag;
    getStop(stop_flag); //读入停止标志对
    Polynomial<Type> L1(stop_flag), L2(stop_flag), L3(stop_flag);
    L1.getPoly(); //读入第一个多项式
    L2.getPoly(); //读入第二个多项式
    L3.addPoly(L1,L2); //L1 = L2 +L3
    L3.dispPloy(); //显示多项式L3的内容
    return 0;
}
```

线性表

- ① 线性表
- ② 顺序表及实现
- ③ 链表及实现
- ④ 一元多项式
- ⑤ 稀疏矩阵
- ⑥ 字符串 *

稀疏矩阵

矩阵中的非零元素个数远远小于矩阵元素总数，并且非零元素的分布没有规律。

- ▶ 稀疏矩阵存储的直观方法是只存储其中的非零元素和非零元素所在的位置。
- ▶ 每个非零元素 a_{ij} 可以用一个三元组来表示： (i, j, a_{ij}) ，然后将此三元组按照一定的次序排列，如先按照行序再按照列序排列。

如：一个二维矩阵可以用一组三元组 $(0,2,5),(0,3,8),(1,0,6),(2,1,5),(2,4,-5)$ 表示

稀疏矩阵

triple

```
struct triple
{
    int row,col;
    int data;
};
```

- ▶ 将三元组作为 elemType 放在顺序表或者链表中。
- ▶ 利用两种不同结构存储稀疏矩阵并分别完成矩阵的加法、乘法、转置任务可以作为课后练习完成。

- ▶ 对于最一般的线性结构-线性表，从逻辑结构 + 基本操作、物理结构、基本操作实现、线性结构典型应用四个方面展开了讨论。这四个方面也是后续讨论任何一种数据结构的方法、脉络。
- ▶ 在逻辑结构和基本操作分析中：用伪代码书写的抽象数据类型来描述结构中元素、元素间关系和日常生活中线性结构的常见基本操作。常见基本操作，源自于人类在生活中的观察和积累。一般分为结构构造类、属性类、数据操纵类、遍历类和典型应用类基本操作。

► 在物理结构分析中：

- ✓ 讨论了将数据存储在内存在连续的空间并利用存储位置的先和后来体现元素关系先后的顺序存储法；
- ✓ 讨论了将元素分别存储在内存在不连续的地方，通过对每个元素附加指针的方法存储元素间关系的链式存储法。
- ✓ 详细讨论了两种结构的不同特征描述，并给出了两种结构的类描述。
- ✓ 在这一阶段，学习者需对本书采用的 C++ 语言的语法进行回顾和复习。

► 在基本操作实现及分析中：

- ✓ 对数据分别在顺序结构和链式结构存储时的常见典型操作，进行了算法设计、代码实现和算法复杂度分析。
- ✓ 掌握利用参数分析、空间检查、核心操作、对其他属性的影响、正确返回的“五步口诀法”，设计一个相对完整的程序。
- ✓ 通过对算法的复杂度分析，了解两种不同存储结构的优缺点和适用场合。

► 典型应用中:

详细讨论了一元多项式、稀疏矩阵的存储和运算。

在现实生活中, 具有线性关系的数据远远不止这些。但通过这三方面的例子, 可以体会到: 通过调用本章分析和建立的线性表 (顺序表、链表), 将这些表作为工具或者利用其存储和处理思路, 可以非常方便地处理部分实际生活中的问题。

线性表

- ① 线性表
- ② 顺序表及实现
- ③ 链表及实现
- ④ 一元多项式
- ⑤ 稀疏矩阵
- ⑥ 字符串 *

字符串简称串

由零个或多个字符组成的有限序列，一般记为 $s = "a_0a_1a_2...a_{n-1}"$ ，($n \geq 0$)。其中 s 称作串名，用双引号括起来的字符序列是串的值。字符 a_i ($0 \leq i \leq n-1$) 可以是字母、数字或其它字符， n 为串的长度。

如： $a = \text{"SHANGHAI"}$ ， a 为串名，串值为 SHANGHAI，串长度为 8。

- 如果把单个字符看作一个元素，整个串看作由多个元素组成的有序序列，和前面的线性表很相似。
- 线性表强调的是单个元素，字符串除了单个元素外，更强调的是一组连续的元素，因此字符串基本操作有一定的特殊性。
- 字符串中元素限定为简单的字符，而线性表中元素为任意类型，甚至可以是复杂的结构。

- ▶ 空串：串的长度为零，但仍然为一个串。
- ▶ 空格串：由一个或一个以上的空格组成的串，串的长度为空格的个数。
- ▶ 单字符串：串中只有一个字符，串长度为 1。
- ▶ 串相等：当且仅当两个串长度相同，且对应位置上的字符完全相同。

- ▶ 子串：一个串中任意个连续的字符组成的子序列称为该串的子串，子串在主串中的位置以子串的第一个字符在主串中的字符位置来表示。
- ▶ 主串：包含子串的串称为主串。

如：“ ” 为空串；“ ” 为长度为 1 的空格串；如果 $a = \text{“SHANGHAI JIAO TONG UNIVERSITY”}$ 、 $b = \text{“JIAO TONG”}$ ，则 b 是子串、 a 为主串、 b 在 a 中的位置为 9。

串的存储

顺序及链式两种存储方式

顺序方式：字符序列在连续的存储空间中依次连续地存放；

链式方式：串中每个字符作为一个结点独立地存放在不连续的存储空间中。

链式存储

- ▶ 方法一是：一个结点中只放单一的字符，为了得到下一个字符结点，还必须有一个指向下一个字符结点的指针。通常一个字符占用一个字节，一个指针却要占用 4 个字节，显然从空间利用的角度，这种存储非常不合算。
- ▶ 方法二是：将字符序列分成若干等长的组，每个组占据一个结点，结点通常不要求一定满，便于个别的插入和删除，且插入和删除时可能意味着结点的分裂和合并，由于串操作常常为对连续的字符子序列进行，操作相对复杂些。（留待课后练习）

考虑以上因素，下面仅讨论，串的顺序存储。

串的存储

顺序存储：一般分静态存储和动态存储两种方式。

- ▶ 静态存储，用一组地址连续的存储单元存储串中的字符序列，该存储空间的大小须预先定义，即使用静态数组，一些串操作结果会因预设空间不够而自动截断。
比如，当结果串存储空间大小为 10 时，若将“SHANGHAI JIAO TONG”赋值给结果串，则结果串串值为“SHANGHAI”，串长度为 9，最后 1 个空间留给串结束符’。
- ▶ 动态存储，同样可以用一组地址连续的存储空间存放字符串，但空间大小可以在程序执行过程中由用户输入，空间在程序运行时动态分配得来，即使用动态数组。经过串操作之后得到的结果字符串全部可以得以保留。

常见错误：

对于字符串“ok”的存储，两个字符加上一个结束标志字符“\0”，共占用了 3 个字节。常见错误是把该字符串长度计为 3，应该是 2，结束符不算入内。

串的基本操作

- ▶ **串的长度**：求串中字符的个数，如“SHANG HAI”长度为 9。
- ▶ **串相等操作**：判断两个字符串是否长度相等，且对应位置上的字符也相等。若二者均满足，返回 true；若有一样不等，返回 false。
如“SHANGHAI”和“SHANGHAI”相等，“SHANGHAI”和“SHANGHAAI ”不等。
- ▶ **赋值操作**：将一个字符串赋值给另一个串。
如 t = “SHANGHAI”, s = “UNIVERSITY”, 将 s 的值赋给 t, 则 t 的值变为“UNIVERSITY”。

串的基本操作

- ▶ **连接操作**：将一个字符串中的字符序列，连接在另一个串字符序列之后，形成一个新的串。

如 $t = \text{"SHANGHAI"}$, $s = \text{"JIAOTONG"}$ 。连接 t 和 s , 即操作 $t += s$ 之后, 得到字符串 $t = \text{"SHANGHAIJIAOTONG"}$; 而连接 s 和 t , 即 $s += t$ 之后得到字符串 $t = \text{"JIAOTONGSHANGHAI"}$ 。

- ▶ **定位操作**：对于一个字符或字符串，求其在另一个字符串中指定字符位置之后首次出现的位置。

如 $t = \text{"SHANGHAI"}$, $s = \text{"HA"}$, 则 s 在 t 的第 3 个字符及其之后首次出现的位置序号为 5。

串的基本操作

- ▶ **求子串操作：** 在一个主串中，从指定的位置序号开始，取得一定长度的字符序列。

如 $t = \text{"SHANGHAI"}$ ，对 t 取第 2 个字符开始的 4 个字符长度的子串为：“ANGH”。

当要求的子串长度过长，超出了主串在指定字符位置后的长度限制，以主串所能提供的最大长度为准。如 $t = \text{"SHANGHAI"}$ ，对 t 取第 2 个字符开始的 10 个字符长度的子串，实际变为取第 2 个字符开始的 6 个字符长度的子串，结果为：“ANGHAI”。

串的基本操作

- ▶ **插入操作：** 在字符串指定的位置上插入另外一个字符串。
如 $t = \text{"SHANGHAI"}$ ，在第 5 个位置上插入字符串 "123" 后得字符串 "SHANG123HAI" 。
- ▶ **删除操作：** 对于一个字符串，从指定的字符位置开始，删除一定长度的字符子序列。
如 "SHANG123HAI" ，从第 5 个字符开始，删除 3 个字符后为 "SHANGHAI" 。

C++ 中关于串的库

- ▶ C++ 有两个字符串处理的库 `cstring.h`(面向过程)、`string`(面向对象), 它们已经提供了许多实现串操作的函数, 其功能和上面提供的基本操作基本相同。
- ▶ 这里用面向对象的方法给出字符串结构的定义、基本操作和其实现, 是为了让大家进一步体会串数据结构的处理本质, 相当于自定义的一个 `string` 库。

字符串类定义 sstring.h

sstring

```
#ifndef SSTRING_H_INCLUDED
#define SSTRING_H_INCLUDED
#include <iostream>
using namespace std;

class illegalSize{};
class outOfBound{};
class noSpace{};
```

```
class sstring
{
    friend int* nextValue(const sstring &t); //
    外部函数，计算失配函数
    private:
        char *str; //动态数组，存储字符串
        int  maxSize; //数组的尺寸
    public:
        sstring(int size); //创建动态空间，数组长
            度 size，字符串长度为0。
        sstring(const char *t); //用字符串 t 初始
            化。
        sstring(const sstring &t); //用同类对象 t
            初始化。
```

字符串类定义 sstring.h

sstring

```
int length() const; //实际存储
                        的字符串长度
void disp() const {cout<<str<<
    endl;}; //显示字符串

//判断两个字符串是否内容一样。
//是返回 true, 否返回 false。
bool equal(const sstring &t)
    const;
void assign(const sstring &t);
    //赋值操作, 将 t 中字符串赋值
    给调用函数的对象
sstring &subString(int pos, int
    len) const; //求从 pos 开
    始, 长度为 len 的子串。

//从串的第 start 个字符起, 向后查找字符串 t 第一次在
//串中出现的位置找到返回位置序号, 未找到返回 -1。
int BF_find(const sstring &t, int start )const; //BF 算法
int KMP_find(const sstring &t, int start )const; //KMP 算法

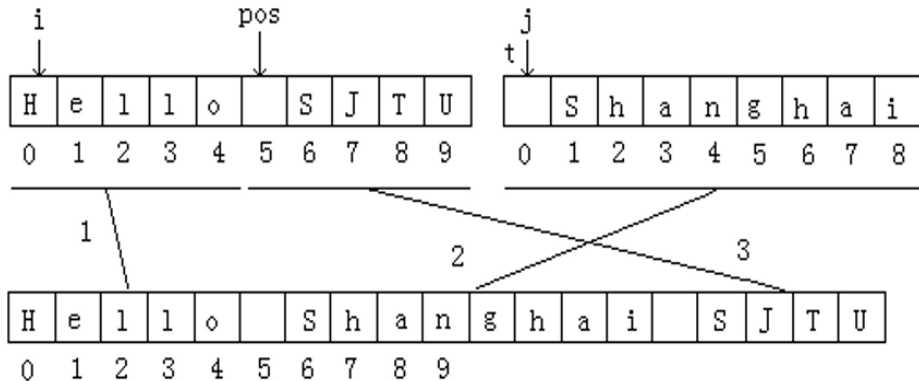
// 在串的第 pos 个字符位置位置上, 插入串 t 的字符串。
// 插入成功返回 true, 否则返回 false。
bool insert(int pos, const sstring &t);
    // 从串的字符串的第 pos 个字符位置起, 删除长度为 n 的子串。
    // 如果长度不够 length, 以实际长度为准。删除成功返回 true, 否
    则返回 false。
bool Remove(int pos, int n);

~sstring(){delete []str;}; //释放动态空间
};
#endif // SSTRING_H_INCLUDED
```


插入操作分析

`bool insert(int pos, const sstring &t):` 将串 `t` 插入到当前串的第 `pos` 个位置中。

如将 “Shanghai ” 插入到 “Hello SJTU” 中，其中 `pos=5`。操作时，先为插入串腾出空间，之后将插入串中字符逐个抄入腾出的空间，插入操作结束。



字符串类部分基本操作的实现 sstring.cpp

sstring

```
#include <iostream>
#include "sstring.h"
using namespace std;
sstring::sstring(int size)//创建动态空间，数组长度size，字符串长度为0。
{
    if (size <= 0) throw illegalSize();
    str = new char[size];//动态申请数组空间
    if (!str)throw noSpace();
    maxSize = size;
    str[0]='\0';
}
```

```
//求串中从pos开始，长度为len的子串。
sstring & sstring::subString(int pos, int len) const
{
    int i;
    if (pos<0) throw outOfBound();

    sstring *tmp = new sstring(len+1);
    for (i=0; i<len; i++)
        if (str[pos+i]=='\0') break;
        else tmp->str[i]=str[pos+i];
    tmp->str[i]='\0';

    return *tmp; }
```

字符串类部分基本操作的实现 sstring.cpp

sstring

//判断两个对象中存储的字符串是否内容一样。是返回 *true*，否返回 *false*。

```
bool sstring::equal(const sstring &t) const
```

```
{
```

```
    int i=0;
```

```
    if (length() !=t.length())
```

```
        return false;
```

```
    while (str[i]!='\0')
```

```
        if (str[i]!=t.str[i]) return false;
```

```
        else i++;
```

```
    return true;
```

```
}
```

```
#include <iostream>
```

```
#include "sstring.h"
```

```
using namespace std;
```

```
int main()
```

```
{    sstring s1(50), s2("abd"), s3(s2), s4("sjtu");
```

```
    s1.disp(); s2.disp(); s3.disp();
```

```
    cout<<"length of s2 is: "<<s2.length()<<endl;
```

```
    if (s2.equal(s3)) cout<<"s2 is equal to s3"<<endl
```

```
    ;
```

```
    if (s1.equal(s3)) cout<<"s1 is equal to s3"<<endl
```

```
    ;
```

```
    s1.assign(s2);    cout<<"after assign s1: "; s1.
```

```
        disp();
```

```
    s1.insert(1,s4);  cout<<"after insert s1: "; s1.
```

```
        disp();
```

```
    s3.Remove(1,2);  cout<<"after remove s3: "; s3.disp
```

```
        ();
```

```
    sstring &s6 = s1.subString(2,4);
```

```
    s6.disp();
```

```
    return 0;
```

```
}
```

常见错误

- ▶ 对于字符串 `s`、`t` 的相互赋值，可能会使用 `s.str=t.str`，但这并非赋值的真正意图。其意图应是：用循环将 `t.str` 指向的数组中所有元素抄写到 `s.str` 指向的数组中，即 `s.str[i]=t.str[i]`。
- ▶ 各种操作都要特别关注结果串中是否保证了“关门字符”`\0` 的存在，这点常常会忽略，但系统并不会报错，输出结果是一个无终止的长长的错误串。

模式匹配

函数 `int find(const sstring &t, int start )`

从字符串（以下称主串 `s`）的第 `start` 个位置的字符起，向后查找字符串 `t` 第一次在主串中出现的位置。如果在主串中找到该子串，返回其首次出现的位置，否则返回-1。

该操作相当于在主串的所有子串集合中匹配待查找的子串，因此该操作也被称作**模式匹配**。

模式匹配

模式匹配 (亦称样品匹配) 是各种串处理中最具有代表性的操作。一般被匹配串 s 称为主串, 匹配串 t 称为模式。

设主串和模式 t 的串长度分别为 n 和 m ; 主串值为 “ $s_0s_1s_2s_3 \dots s_{n-1}$ ”, 模式值为 “ $t_0t_1t_2t_3 \dots t_{m-1}$ ”, 起始位置 $start = 0$

模式匹配的两种算法:

Brute-Force(BF) 算法

Knuth-Morris-Pratt(KMP) 算法

Brute-Force(BF) 算法思路:

以 $s = \text{"SHANGHAI"}$, $t = \text{"HAI"}$ 为例, $n=8$, $m=3$ 。 i 指示主串当前字符的下标, j 指示模式当前字符的下标。

第一轮匹配 S H A N G H A I 起始: $i=0$, $j=0$
 H A I 结束: $i=0$, $j=0$

第二轮匹配 S H A N G H A I 起始: i=1, j=0
 |||
 H A I 结束: i=3, j=2

第三轮匹配 S H A N G H A I 起始: i=2, j=0
 ||
 H A I 结束: i=2, j=0

第六轮匹配 S H A N G H A I 起始: i=5, j=0
H A I 结束: i=7, j=2

Brute-Force(BF) 算法实现

BF

//从串的第 *start* 个字符起，向后查找字符串 *t* 第一次在
串中出现的位置找到返回位置序号，未找到返回 -1。

```
int sstring::BF_find(const sstring &t, int start) const
{
    int curStart; //每次比较时，主串的起始位置
    int i, j=0;    int pos=-1; //匹配不成功返回 -1
    int n, m;
    n = length(); m = t.length(); //主串长度和模式长度
    curStart=start;
```

```
while (curStart <= (n-m)) //剩余主串结点长度大于等于模式串长度
{
    i=curStart;
    while ((j<m)&&(str[i]==t.str[j])) {
        i++; j++;
    }
    if (j==m) {pos = curStart; break; } //匹配成功返回位置
    curStart++; j=0;
}
return pos;
}
```


Brute-Force(BF) 算法分析

最好情况：

主串从 0 下标开始比较整个模式即告成功，时间为 $O(m)$

主串中每个位置上的字符都和模式中首字符不等，时间为 $O(n)$

最差情况：主串从 0 下标一直到 $n-m$ 下标开始和模式比较时，每次都是模式的最后一个字符不等，结果为匹配不成功；或者只有当主串从下标为 $n-m$ 开始和模式比较相等，结果为匹配成功，时间均为 $O((n-m)m)$ 。

Knuth-Morris-Pratt(KMP) 算法

问题的引入：

匹配时，前 4 个元素相同，第 5 个不同。因模式中元素各不相同，故下次可以让模式的第 1 个字符和主串的第 5 个字符开始比较，中间可以省略。

第一轮匹配：

起始：i=0, j=0

主串：A B C D E F G H I J K

|| || || || ✕

模式：A B C D T

第二轮匹配：

起始：i=4, j=0

主串：A B C D E F G H I J K

|

模式：A B C D T

根据模式中字符的特征 + 已经和主串中比较的结果，决定下次比较时主串的起始位置。

Knuth-Morris-Pratt(KMP) 算法

问题的引入：

匹配时，前 4 个元素相同，第 5 个不同。因模式中前 4 个元素均相同，故下次可以让模式的第 4 个字符和主串的第 5 个字符开始比较，中间可以省略。

第一轮匹配：

起始： $i=0$, $j=0$

主串： A A A A A B C D E F G

 || || || || ✖
模式： A A A A B

第二轮匹配：

起始： $i=5$, $j=0$

主串： A A A A A B C D E F G

 |
模式： A A A A B

根据模式中字符的特征 + 已经和主串中比较的结果，决定下次比较时模式的起始位置。

Knuth-Morris-Pratt(KMP) 算法

问题的引入:

观察第三个例子

s: a b c a b c a b c d
 ↓

t: a b c a b c d

(1)

s: a b c a b c a b c d
 ↓

t: a b c a b c d

(2)

s: a b c a b c a b c d
 ↓

t: a b c a b c d

(3)

s: a b c a b c a b c d
 ~~###~~ || || || ||

t: a b c a b c d

(4)

Knuth-Morris-Pratt(KMP) 算法

相关概念:

前缀: 一个串的前缀, 是指从第 1 个字符开始到它的任意一个字符为止的子串。

对模式 $t = t_0t_1t_2t_3 \cdots t_m$ 而言, $t_0t_1t_2$ 就是它的长度为 3 的前缀, $t_0t_1t_2t_3t_4$ 就是它的长度为 5 的前缀。

失配: 当主串中某字符和模式中某字符比较且不等时, 称发生失配。主串中发生失配的位置称失配点。

Knuth-Morris-Pratt(KMP) 算法

s: a b c a b c a b c d
t: a b c a b c d

最长前缀：图中， $s_6 \neq t_6$ ，发生失配。观察以模式失配点 t_6 的前一字符 t_5 为结束字符的子串， $t_0t_1t_2$ 是它的一个前缀，而失配点 t_6 前有一个子串 $t_3t_4t_5$ ，有 $t_3t_4t_5 = t_0t_1t_2$ 关系存在，称 $t_0t_1t_2$ 为该子串的最长前缀。假如模式串为 $r = \text{"abcdsytuabf"}$ ，则对字符 f 前的子串来说，最长前缀为 r_0r_1 即 "ab" 。

上图中，第二轮匹配可直接将主串失配点 s_6 和模式最长前缀 $t_0t_1t_2$ 的后一字符 t_3 进行比较即可。由于 $t_3 = s_6$ ， $t_4 = s_7$ ， $t_5 = s_8$ ， $t_6 = s_9$ ，这样就在主串中找到了模式 t 。

Knuth-Morris-Pratt(KMP) 算法

最长前缀获取：在实际应用中，主串通常比模式长得多，因此在模式匹配开始之前，值得花点时间将模式中字符的情况，完全分析清楚，因此这个任务是模式匹配前的首要任务。

失配函数：对模式中每一个位置上的字符，发现从开始到该位置前一字符形成的子串的最长前缀。下面用一个整型数组 $\text{next}[j]$ 记录模式中第 j 个字符前子串的最长前缀的长度，以后将 next 称为失配函数。

```
s:  a b c a b c a b c d
      |
t:  a b c a b c d
```

Knuth-Morris-Pratt(KMP) 算法

计算失配函数的算法：

nextValue

//返回模式 *t* 的失配函数

```
int* nextValue(const sstring &t)
{
    int *next, m, i, j, k;
    m = t.length();
    if (m==0) return NULL;
    next = new int[m];
    next[0]=-1; next[1]=0;
```

```
s: a b c a b c a b c d
      ↑
t: a b c a b c d
```

```
for (j=2; j<m; j++)
{
    for (k=j-1; k>0; k--)
    {
        for (i=0; i<k; i++)
            if (t.str[i]!=t.str[j-k+i]) break;
        if (i==k) break;
    }
    if (k==0)
        next[j] = 0;
    else
        next[j] = k;
}
return next;
}
```


Knuth-Morris-Pratt(KMP) 算法思路

假设主串 s 和模式 t 在匹配过程中, 发生了失配, 即 t_j 和 s_i 不等。

假设以模式失配点的前一字符 t_{j-1} 为结束字符的子串的最长前缀长度为 k , 则主串 s 的失配点 s_i 下一步只需和 t_k 继续比较下去就可以了。若 $t_k = s_i$, 则继续比较 t_{k+1} 和 s_{i+1} , 如此继续进行, 直至找到该模式, 或者断定该模式不存在为止。

Knuth-Morris-Pratt(KMP) 算法

计算失配函数的算法：

KMP

```
//从串的第 start 个字符起，向后  
//查找字符串 t 第一次在串中出现  
//的位置找到返回位置序号，未  
//找到返回 -1。  
int sstring::KMP_find(const sstring &t, int  
    start )const  
{  
    int n = length(), m = t.length();  
    int *next, curStart;  
    int i,j;  
    int pos=-1;  
    next = nextValue(t);
```

```
//根据 next 数组中的值在主串中查找模式  
i=start; j=0;  
while (i<=(n-m))  
{    curStart = i-j;  
    while ((j<m)&&(str[i]==t.str[j])){i  
        ++;j++;}  
    if (j==m)  
    {    pos=curStart;    break;    }  
    else  
    {    if (next[j]==-1) {++i;j=0;}  
        else {j=next[j];}  
    }  
}  
return pos;  
}
```

Knuth-Morris-Pratt(KMP) 算法分析

计算失配函数，时间达 $O(m^3)$ 。

KMP 算法，观察主串的指针 i ：

i 开始为 0，每次比较后， i 不会减少，它或者不变或者增大 1，直至 $n-m$ 。

i 不变的次数共有多少？

如果本次开始匹配主串的第 i 个字符和模式中第 1 个字符相等，则 i 增加 1；如果不等，本次模式匹配结束，下次模式匹配时 i 也增加 1，故主串中每个 i 最多用 2 次，总体有 $2(n-m)$ 次比较。

特殊地，如果模式的第 1 个字符和主串的第 1 个字符比较时总是不等，比较次数最少，为 $n-m$ 次。

Knuth-Morris-Pratt(KMP) 算法的优点

在硬盘当中寻找模式时，主串指针 i 不会变小（即不回溯），是一个很大的优点。

设内存保存了模式和 `next` 数组，还保存了从硬盘读来的主串中一个扇区大小的字符串。

假设第一个扇区的字符串同模式的相应字符串完全匹配了，下一步将读入第二个扇区的字符串到内存缓冲区，将第一个扇区的字符串覆盖掉；假设已在内存缓冲区中的第二个扇区的字符和模式进行匹配时，发生失配，用 BF 算法，就要将第一个扇区的字符串重新读入到内存缓冲区才可以继续匹配下去，重新读入非常费时，而 KMP 算法完全避免了这个缺点，是一个很好的算法。

- ▶ 对于最一般的线性结构-线性表，从逻辑结构、物理结构、基本操作实现、线性结构典型应用四个方面展开了讨论。这四个方面也是后续讨论任何一种数据结构的方法、脉络。
- ▶ 在逻辑结构和基本操作分析中：用伪代码书写的抽象数据类型来描述结构中元素、元素间关系和日常生活中线性结构的常见基本操作。常见基本操作，源自于人类在生活中的观察和积累。一般分为结构构造类、属性类、数据操纵类、遍历类和典型应用类基本操作。

► 在物理结构分析中：

- ✓ 讨论了将数据存储在内存在连续的空间并利用存储位置的先和后来体现元素关系先后的顺序存储法；
- ✓ 讨论了将元素分别存储在内存在不连续的地方，通过对每个元素附加指针的方法存储元素间关系的链式存储法。
- ✓ 详细讨论了两种结构的不同特征描述，并给出了两种结构的类描述。
- ✓ 在这一阶段，学习者需对本书采用的 C++ 语言的语法进行回顾和复习。

► 在基本操作实现及分析中：

- ✓ 对数据分别在顺序结构和链式结构存储时的常见典型操作，进行了算法设计、代码实现和算法复杂度分析。
- ✓ 掌握利用参数分析、空间检查、核心操作、对其他属性的影响、正确返回的“五步口诀法”，设计一个相对完整的程序。
- ✓ 通过对算法的复杂度分析，了解两种不同存储结构的优缺点和适用场合。

► 典型应用中:

详细讨论了一元多项式、稀疏矩阵的存储和运算。

在现实生活中，具有线性关系的数据远远不止这些。但通过这三方面的例子，可以体会到：通过调用本章分析和建立的线性表（顺序表、链表），将这些表作为工具或者利用其存储和处理思路，可以非常方便地处理部分实际生活中的问题。

- Polynomial 类中各个成员函数的实现。

扩展：标准模板库中实现

- STL（标准模版库）是 C++ 中数据结构的实现。这些数据结构被称为集合或容器。
- STL 中线性表的实现有两种：
 - Vector：线性表的顺序实现
 - List：线性表的双链表的实现

目标

- 熟练 C++ **面向对象** 的编程方法、C++ 语法、书写完整程序的技巧。
- 接触并逐步熟悉研究一种数据结构的角度和方法。
- 掌握 **顺序结构** 的基本功。
- 掌握 **链式操作** 的基本功。
- 掌握 **数据结构工具** 的建造和工具使用的不同。
- 掌握最基本且简单的 **线性结构-线性表** 结构

- 以下哪个操作在单链表上实现比顺序表实现效率高？
 - A. 在最后一个元素的后面插入一个新元素
 - B. 顺序输出前 k 个元素
 - C. 删除所有值为 x 的元素
 - D. 交换表中两个元素

- 以下哪个操作在单链表上实现比顺序表实现效率高？ C
 - A. 在最后一个元素的后面插入一个新元素
 - B. 顺序输出前 k 个元素
 - C. 删除所有值为 x 的元素
 - D. 交换表中两个元素